

Application Layer - Comprehensive Theory Notes

1. Application Layer Overview

Introduction to Application Layer

The Application Layer is the topmost layer in the Internet protocol stack, serving as the interface between network applications and the underlying transport services. This layer is where network applications and their application-layer protocols reside, providing services directly to users.

- **Popular application protocols** - HTTP, SMTP, IMAP, DNS

Common Network Applications

Modern network applications encompass a vast range of services that users interact with daily. These applications demonstrate the practical importance of application-layer protocols and services.

Popular Network Applications:

- **Social networking platforms** - Facebook, Twitter, Instagram
- **Web browsing** - HTTP-based content delivery
- **Communication services** - Email, text messaging, video conferencing
- **Entertainment** - Streaming video (YouTube, Netflix), online gaming
- **File sharing** - P2P applications, cloud storage
- **Voice services** - VoIP applications like Skype
- **Search engines** - Google, Bing, Yahoo
- **Remote access** - SSH, Telnet, remote desktop protocols

2. Network Application Architectures

Client-Server Architecture

Client-server architecture is a common way of building network applications. In this system, the **server** is the computer that provides services (like storing files, hosting a website, or handling requests), and the **client** is the computer that uses those services. The roles are clearly separated: servers serve, and clients request or consume.

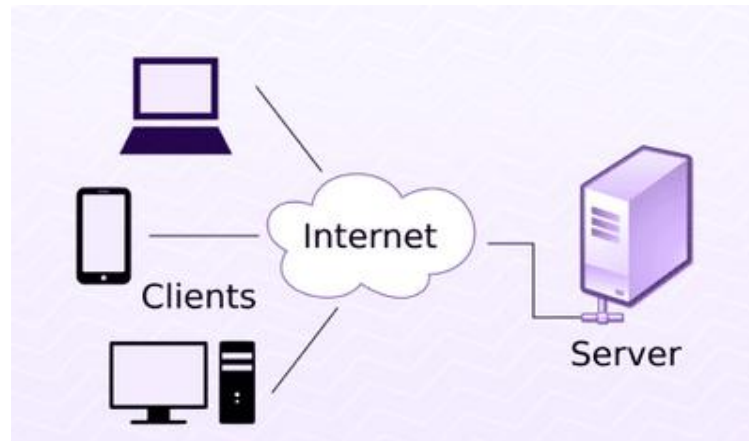
Server Characteristics:

- **Always-on host** - Servers must be continuously available to respond to client requests
- **Permanent IP address** - Servers maintain fixed IP addresses for reliable client access
- **Data center deployment** - Often deployed in data centers for scalability and reliability

- **Centralized control** - All application logic and data management occurs at the server

Client Characteristics:

- **Intermittent connectivity** - Clients connect to servers only when needed
- **Dynamic IP addresses** - Clients may have changing IP addresses (DHCP)
- **No direct communication** - Clients do not communicate directly with each other
- **Service consumers** - Clients request and consume services provided by servers



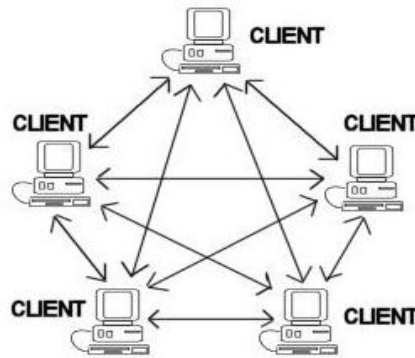
Examples and Applications: The client-server model is exemplified by protocols such as HTTP (web browsing), IMAP (email access), and FTP (file transfer). In each case, clients initiate communication with servers that provide the requested services.

Peer-to-Peer (P2P) Architecture

Peer-to-peer (P2P) architecture is a system where computers (called peers) connect and talk directly with each other, instead of always relying on a central server. In this model, every peer can both provide services and use services. This makes the system more decentralized and doesn't require one computer to always stay on as the main server.

Peer-to-Peer (P2P) Characteristics:

- **No fixed server** – There's no single computer that must always stay on. This avoids failures or overload at one point.
- **Direct communication** – Computers (peers) talk to each other directly without needing a middle server.
- **Both roles** – Every peer can work as a **client** (using services) and as a **server** (providing services).
- **Grows by itself** – When new peers join, they add extra resources (like storage or processing power) while also using the network.
- **Flexible joining** – Peers can join or leave the network anytime without breaking it.



Advantages of P2P: The primary advantage of P2P architecture is its self-scalability. As new peers join the network, they contribute both computational resources and bandwidth while also generating new service demands. This creates a naturally scaling system where increased demand is matched by increased capacity.

Challenges in P2P:

- **Intermittent connectivity** - Peers may leave the network unpredictably
- **Changing IP addresses** - Peer identification becomes complex
- **Security concerns** - Distributed nature makes security management difficult
- **Quality of service** - Difficult to guarantee consistent service quality

P2P Applications: Examples include BitTorrent for file sharing, Skype for VoIP communication, and various streaming applications that distribute content delivery among participating peers.

3. Process Communication

Understanding Processes in Network Communication

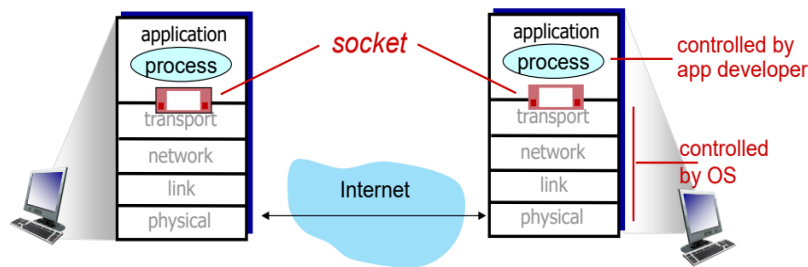
Network communication fundamentally occurs between processes running on different host systems. A process is defined as a program running within a host, and understanding how processes communicate across networks is essential for application development.

Local vs. Network Communication:

- **Same host communication** - Processes on the same host communicate using **inter-process communication (IPC) mechanisms** defined by the operating system
- **Different host communication** - Processes on different hosts communicate **by exchanging messages through the network**
- **Process roles** - Client processes initiate communication, while server processes wait to be contacted

Socket Interface

A socket is like a communication endpoint that allows two programs (usually running on different computers) to send and receive data over a network. Sockets serve as the programming interface **between application processes and the underlying transport protocol**. They can be conceptualized as doors through which application processes send and receive network messages.



Socket Functionality: The socket acts as an intermediary between the application layer and the transport layer. When a process sends a message, it pushes the message through its socket, relying on the transport infrastructure to deliver it to the receiving process's socket.

Socket Communication Model:

- **Sending process** - Pushes messages through its socket
- **Transport infrastructure** - Handles message delivery across the network
- **Receiving process** - Receives messages from its socket
- **Bidirectional communication** - Two sockets are involved, one on each communicating host

Process Addressing

For processes to communicate across networks, they must be uniquely identifiable. This addressing system ensures that messages reach their intended destinations.

IP Address Limitations: While each host has a unique 32-bit IP address, this alone is insufficient for identifying specific processes because multiple processes can run simultaneously on a single host.

Port Numbers: The solution involves using port numbers in conjunction with IP addresses. A complete process identifier includes both the IP address of the host and the port number associated with the specific process.

Standard Port Numbers:

- **HTTP server** - Port 80
- **HTTPS server** - Port 443
- **Mail server (SMTP)** - Port 25
- **DNS server** - Port 53
- **FTP server** - Port 21

Example Addressing: To send an HTTP message to the web server at gaia.cs.umass.edu, a client would use IP address 128.119.245.12 and port number 80.

Application-Layer Protocols

Application-layer protocols define the rules and formats for communication between application processes. These protocols specify the structure and semantics of messages exchanged between applications.

Protocol Components:

- **Message types** - Request and response message categories
- **Message syntax** - Field definitions and delimiters within messages
- **Message semantics** - It means the **meaning and purpose** of the information contained in the message fields
- **Communication rules** - When and how processes send and respond to messages

Protocol Categories:

Open Protocols:

- Defined in Request for Comments (RFC) documents
- Publicly available specifications
- Enable interoperability between different implementations
- Examples: HTTP, SMTP, DNS

Proprietary Protocols:

- Privately controlled specifications
- May limit interoperability
- Examples: Skype protocol, various gaming protocols

4. Transport Service Requirements

1. Data Integrity

- Some applications require 100% reliable data transfer.
 - Examples: file transfer, web transactions.
- Other applications can tolerate some data loss.
 - Examples: audio or video streaming.

2. Timing (Delay Sensitivity)

- Some applications need low delay to function effectively.
 - Examples: Internet telephony, interactive games
- High delays can make these applications unusable.

3. Throughput

- Certain applications require a minimum guaranteed throughput to function properly.
 - Examples: multimedia streaming

- Other applications, called elastic applications, can work with whatever throughput is available.
 - Examples: email, file download

4. Security

- Some applications require encryption, data integrity, and authentication to protect data.
 - Examples: online banking, confidential transactions.

Internet Transport Protocols: TCP and UDP

1. TCP (Transmission Control Protocol)

Services provided:

- **Reliable transport** between sending and receiving processes.
- **Flow control:** prevents sender from overwhelming receiver.
- **Congestion control:** throttles sender if the network is overloaded.
- **Connection-oriented:** a connection must be **established** before data transfer.

Not provided:

- Timing guarantees
- Minimum throughput guarantee
- Security (encryption, authentication)

2. UDP (User Datagram Protocol)

Services provided:

- **Unreliable transport:** sends messages (datagrams) without guaranteeing delivery.

Not provided:

- Reliability, flow control, congestion control
- Timing, throughput guarantee
- Security
- Connection setup

Q: Why use UDP?

- **Low overhead:** no connection setup or reliability checks → faster.
- **Timing-sensitive applications:** real-time audio/video streaming, online games, VoIP.

- These apps prefer **low delay** over guaranteed delivery.
- **Simple query-response protocols:** e.g., DNS requests, where speed matters and occasional loss is acceptable

5. Web and HTTP

Web Page Structure: A web page consists of multiple objects, each potentially stored on different web servers. Objects can include HTML files, JPEG images, Java applets, audio files, CSS stylesheets, and JavaScript files. The base HTML file contains references to other objects, each addressable by a Uniform Resource Locator (URL).

URL Structure: URLs have two main components:

- **Hostname** - Identifies the server (e.g., www.someschool.edu)
- **Path name** - Specifies the object location (e.g., /someDept/pic.gif)

HTTP Overview

It is the **communication protocol** used for transferring data (like web pages, images, videos, etc.) between a **web browser (client)** and a **web server** over the internet. HTTP (Hyper Text Transfer Protocol) serves as the web's application-layer protocol, implementing a client-server model for web communication.

HTTP Client-Server Model:

- **Client (Browser)** - Requests, receives, and displays web objects using HTTP protocol
- **Server (Web Server)** - Sends objects in response to HTTP requests
- **Examples** - Firefox browser communicating with Apache web server



HTTP Protocol Characteristics: HTTP utilizes TCP as its underlying transport protocol, providing reliable data transfer between clients and servers. The protocol follows a request-response pattern where clients initiate connections and servers respond with requested content.

HTTP Connection Process:

1. **TCP Connection Establishment** - Client creates TCP connection to server port 80
2. **Connection Acceptance** - Server accepts the TCP connection

3. **Message Exchange** - HTTP messages are exchanged between browser and server
4. **Connection Termination** - TCP connection is closed after message exchange

Stateless Nature: HTTP is fundamentally stateless, meaning servers maintain no information about past client requests. This design simplifies server implementation and improves scalability, but creates challenges for applications requiring persistent state information.

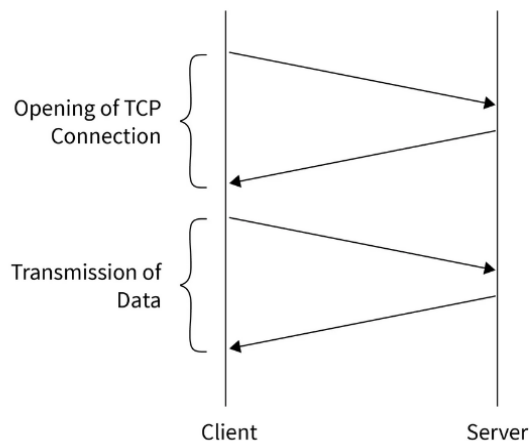
HTTP Connection Types

HTTP supports two different connection models, each with distinct advantages and use cases.

Non-Persistent HTTP (HTTP/1.0): In non-persistent HTTP, each TCP connection handles at most one object transfer. For every additional object (image, CSS, script, etc.), a **new connection** must be established.

Steps in Non-Persistent HTTP Process

1. **Client initiates TCP connection** to the server (e.g., port 80).
2. **Client sends an HTTP request** (e.g., GET request) for a web object.
3. **Server sends back the HTTP response** with the requested object.
4. **Server closes the TCP connection.**
5. If the page has multiple objects (like 1 HTML + 5 images):
 - Client repeats steps 1–4 for each object.



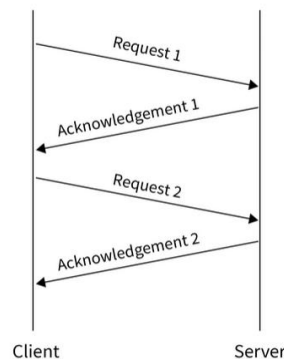
Non-Persistent HTTP Disadvantages:

- **Multiple RTTs** - Requires 2 RTTs per object (connection + request/response)
- **OS overhead** - Each TCP connection requires system resources
- **Performance impact** - Connection establishment delay for each object

Persistent HTTP (HTTP/1.1): Persistent HTTP allows multiple objects to be sent over a single TCP connection.

Steps in Persistent HTTP Process

1. **Client initiates one TCP connection** to the server (port 80 for HTTP, 443 for HTTPS).
2. **Client sends HTTP request** for the first object (e.g., HTML).
3. **Server sends back the HTTP response** with the HTML.
4. Without closing the connection, the **client sends more requests** (for images, CSS, etc.) over the same connection.
5. The connection can remain open for a certain time (timeout) to allow future requests.
6. Eventually, either client or server closes the TCP connection.



Persistent HTTP Advantages:

- **Reduced latency** - As little as one RTT for all referenced objects
- **Resource efficiency** - Fewer TCP connections reduce server load
- **Pipeline support** - Multiple requests can be sent without waiting for responses

HTTP Message Format

HTTP messages follow a standardized format for both requests and responses, using ASCII encoding for human readability.

1. HTTP Request

Structure: Request line + Headers + Optional Body

Request Line Components:

- **Method:** GET, POST, HEAD, PUT, DELETE
- **URL:** Resource path
- **Version:** HTTP/1.1, HTTP/2

Important Headers:

- **Host:** Server address (required)
- **User-Agent:** Browser or client info
- **Accept / Accept-Language / Accept-Encoding:** What content/language/compression is accepted
- **Connection:** keep-alive / close

HTTP Methods:

1. GET

- Used to **get information** from the server.
- Sends data in the **URL** (after ?).
- **Idempotent:** Doing it multiple times has the same result.
- **Cacheable:** Responses can be saved and reused.

2. POST

- Used to **send data** to the server for processing.
- Sends data in the **message body**.
- **Non-idempotent:** Doing it multiple times may create different results.
- Common for **form submissions** (like login, signup).

3. HEAD

- Used to **get only the headers** without the content.
- Helps **check if a resource exists** or get metadata.
- **Saves bandwidth** because no content is sent.

4. PUT

- Used to **upload or replace** a resource on the server.
- **Replaces the entire content** of the resource.
- **Idempotent:** Doing it multiple times has the same effect.

2.HTTP Response Messages

Purpose:

HTTP response messages tell the client the result of its request and deliver the requested content if available.

Structure:

1. **Status Line:**
 - Protocol version (e.g., HTTP/1.1)
 - Status code (numeric)
 - Status phrase (short description)
2. **Headers:** Additional information about the response (e.g., content type, server info).
3. **Optional Body:** The actual content requested (like a webpage, image, or data).

Common Status Codes:

2xx Success Codes:

- **200 OK** - Request succeeded, content follows
- **201 Created** - Resource successfully created
- **204 No Content** - Request succeeded, no content returned

3xx Redirection Codes:

- **301 Moved Permanently** - Resource permanently relocated
- **302 Found** - Resource temporarily relocated
- **304 Not Modified** - Cached content is still valid

4xx Client Error Codes:

- **400 Bad Request** - Malformed request syntax
- **401 Unauthorized** - Authentication required
- **403 Forbidden** - Access denied
- **404 Not Found** - Resource doesn't exist

5xx Server Error Codes:

- **500 Internal Server Error** - Server encountered unexpected condition
- **503 Service Unavailable** - Server temporarily overloaded

Maintaining State: Cookies

A **cookie** is a small piece of data that a website stores on a computer or device to remember information about a user.

- It helps the website maintain information across multiple visits.
- Examples include staying logged in, keeping items in a shopping cart, or saving language preferences.

How Cookies Work:

1. **Response header:** Server sends a Set-Cookie header to the client.
2. **Request header:** Client sends the cookie back in the Cookie header for future requests.
3. **Client storage:** Browser saves the cookie on the user's device.
4. **Server database:** Server keeps a record of cookies to recognize users.

Step-by-Step Cookie Process:

1. User visits a website for the first time — no cookies exist.
2. Server creates a cookie with a unique ID and stores it in its database.
3. Browser saves the cookie locally.
4. In future visits, browser sends the cookie with every request to the site.
5. Server uses the cookie to identify the user and keep track of their session.

Uses of Cookies:

- **User login:** Keeps you logged in across pages.
- **Shopping carts:** Saves items you added to your cart.
- **Personalization:** Shows content based on your preferences.
- **Tracking:** Helps analyze user behavior.

Privacy Points:

- Cookies can track your activity on websites.
- Third-party cookies can track users across multiple sites.
- Modern browsers let users manage or block cookies.

Web Caching and Proxy Servers

Web caching is a technique to make websites load faster by storing frequently used content closer to users. This reduces the need to fetch data from the original server every time.

How a Web Cache Works:

1. A user requests a web page.
2. The cache checks if it already has a copy of the requested content.
 - **Cache hit:** The cache sends the content directly to the user.
 - **Cache miss:** The cache requests the content from the original server, stores it, and then sends it to the user.

Benefits of Caching:

- Speeds up response time
- Reduces network traffic
- Balances server load
- Saves bandwidth costs

Conditional GET (Keeping Cache Updated):

1. Cache sends If-Modified-Since with the last saved timestamp.
2. Server checks if content has changed.
3. **Not changed:** Responds 304 Not Modified (no new content).
4. **Changed:** Responds 200 OK with new content.
5. Cache updates its stored copy if new content is received.

HTTP/2 Improvements

HTTP/2 was created to fix some performance problems of HTTP/1.1, especially for web pages that load many objects like images, scripts, or videos.

Problems in HTTP/1.1:

1. **Head-of-line blocking:** Large files can delay smaller files in the queue.
2. **Sequential processing:** Files are sent one by one in the order they arrive.
3. **Connection overhead:** Multiple connections are needed to speed up loading, which is inefficient.

Improvements in HTTP/2:

1. **Multiplexing:** Multiple files can be sent at the same time over a single connection.
2. **Priority system:** Important files can be prioritized so they load first.
3. **Server push:** The server can send files to the browser even before they are requested.
4. **Frame division:** Each file is divided into small pieces (frames) so different files can be sent together efficiently.

HTTP/3:

- Uses **UDP** instead of TCP to make loading even faster.
- Removes head-of-line blocking at the transport level.
- Provides better error handling and congestion control for each file.

In short: HTTP/2 and HTTP/3 make websites load faster, especially pages with lots of files, by sending multiple files at once and managing them more efficiently.

6. Electronic Mail (SMTP, IMAP)

Email System Architecture

Electronic mail represents one of the earliest and most enduring network applications. The email system architecture consists of three major components that work together to provide reliable message delivery services.

Email System Components:

1. User Agent (UA)

- Software that **allows the user to interact** with the email system.
- Used for **composing, sending, and reading emails**.
- Examples: Gmail (web client), Outlook, Thunderbird.

2. Mail Server

- Works like a **post office**:
 - Stores **incoming mails** in a **mailbox**.
 - Sends **outgoing mails** using **SMTP**.
- Each user has a **mailbox** on the server.
- Mail servers run **MTA (Message Transfer Agent)** software (like Sendmail, Postfix).

Simple Mail Transfer Protocol (SMTP): SMTP serves as the communication protocol between mail servers, ensuring reliable message delivery across the Internet.

SMTP Protocol Details

SMTP operates as a push protocol, meaning the sending server initiates the connection and pushes messages to the receiving server. This differs from protocols like HTTP, which operate on a pull model.

SMTP Characteristics:

- **TCP-based** - Uses TCP port 25 for reliable message transfer
- **Direct transfer** - Sending server connects directly to receiving server
- **ASCII text** - Messages must be encoded in 7-bit ASCII
- **Persistent connections** - Maintains connections for multiple message transfers
- **Command-response interaction** - Similar to HTTP with ASCII commands and numeric responses

SMTP vs HTTP (Comparison)

Feature	SMTP	HTTP
Purpose	Transfer of email messages between mail servers	Transfer of web documents (web pages, files)
Communication Style	Push (sender initiates transfer)	Pull (client requests from server)
Connection Type	Persistent (can send multiple mails over one connection)	Non-persistent (one request per connection) or Persistent (HTTP/1.1 onwards)
Message Format	Requires 7-bit ASCII (non-text encoded)	Binary data allowed directly
Protocol Role	Application-layer protocol for email	Application-layer protocol for web
Ports Used	Port 25 (default), 587/465 (modern usage)	Port 80 (HTTP), 443 (HTTPS)

SMTP Communication Phases:

Handshaking Phase: The communication starts with an exchange of greeting messages. In this step, servers (or end systems) introduce themselves, verify each other's identity, and set up the rules needed to begin the session.

Transfer Phase: During this phase, the actual message content is transferred from sender to receiver, including all header information and message body.

Closure Phase: The communication session ends in an orderly way. Both servers (or end systems) confirm that all messages have been successfully exchanged, and then they acknowledge the termination of the connection.

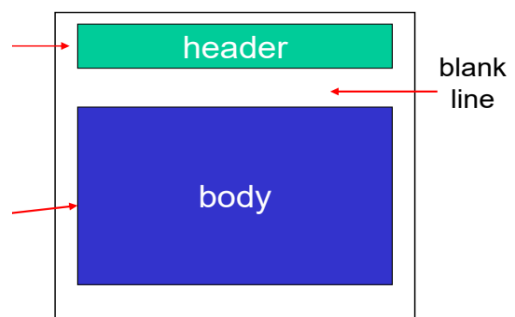
Message Format and Standards

Email message format is defined by RFC 822, which specifies the structure and syntax of email messages, separate from the SMTP protocol used for message transfer.

Message Structure: Email messages consist of two main parts: headers and body, separated by a blank line.

Header Fields:

- **To:** - Primary recipients
- **From:** - Message sender
- **Subject:** - Message topic
- **Date:** - Message creation timestamp
- **Cc:** - Carbon copy recipients
- **Bcc:** - Blind carbon copy recipients



Message Body: The message body contains the actual content and **must be encoded in 7-bit ASCII**. Modern extensions like MIME (Multipurpose Internet Mail Extensions) allow for rich content including attachments, images, and various character encodings.

Mail Access Protocols

Since SMTP is designed for server-to-server communication, additional protocols are needed for users to access their mailboxes stored on mail servers.

Internet Message Access Protocol (IMAP): IMAP provides sophisticated mailbox access capabilities, allowing users to manage email stored on remote servers.

Post Office Protocol (POP): POP provides a simpler mailbox access model designed for single-device email access.

Web-based Email: Modern email services like Gmail, Hotmail, and Yahoo Mail provide web-based interfaces that combine SMTP for sending with IMAP or POP for message retrieval, all accessible through standard web browsers using HTTP/HTTPS.

7. Domain Name System (DNS)

DNS Fundamentals and Purpose

The Domain Name System represents one of the most critical infrastructure services on the Internet, **providing the essential mapping between human-readable domain names and machine-readable IP addresses**. Without DNS, users would need to memorize numeric IP addresses to access websites and services.

The Naming Problem: Internet hosts and routers are identified by 32-bit IP addresses used for routing datagrams, but humans prefer memorable names like cs.umass.edu. DNS solves this fundamental mismatch between human preferences and network requirements.

DNS as a Distributed System: DNS is implemented as a distributed database across a hierarchy of name servers worldwide. This distributed approach ensures scalability, reliability, and performance while avoiding the problems that would arise from a centralized system.

Why Not Centralize DNS: A centralized DNS system would suffer from several critical problems:

- **Single point of failure** - Entire Internet would be vulnerable to single server failure
- **Traffic volume** - Single server couldn't handle billions of queries
- **Distant centralized database** - Long delays for users far from the server
- **Maintenance complexity** - Impossible to manage all Internet names from one location

DNS Services and Functionality

DNS provides multiple services beyond simple hostname-to-IP address translation, supporting various Internet applications and services.

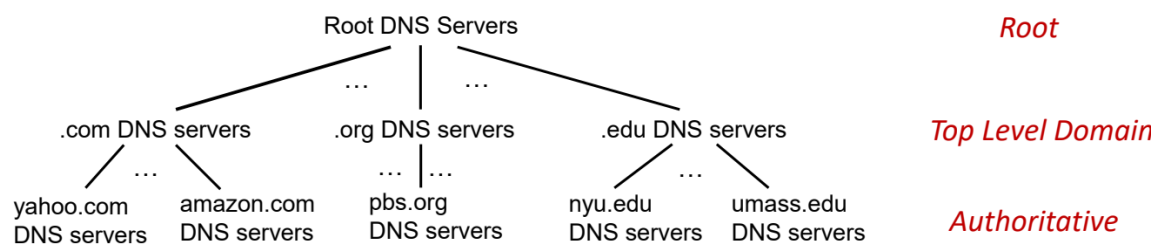
Core DNS Services:

- **Hostname to IP address translation** - Primary function enabling human-friendly naming
- **Host aliasing** - Single host can have multiple names (canonical and alias names)
- **Mail server aliasing** - Special handling for email server identification
- **Load distribution** - Multiple IP addresses for replicated web servers

DNS Query Volume: The scale of DNS operation is enormous, with major providers like Comcast handling over 600 billion DNS queries per day, demonstrating the critical importance of efficient DNS infrastructure.

Hierarchy and Structure

DNS implements a hierarchical naming structure that reflects administrative boundaries and provides efficient query resolution.



1. Root Level

- **Top of DNS hierarchy**, managed by **root DNS servers**.
- Provides **authoritative information about TLDs**.
- Key Points:
 - **13 logical root servers** – distributed worldwide with multiple physical instances
 - **Critical function** – Internet cannot function without root servers
 - **DNSSEC support** – ensures authentication and message integrity
 - **ICANN management** – overseen by Internet Corporation for Assigned Names and Numbers

2. Top-Level Domain (TLD) Servers

- Responsible for **major domain categories and country codes**.
- Types:
 - **Generic TLDs (gTLDs)**: .com, .org, .net, .edu, .aero, .jobs, .museums

- **Country Code TLDs (ccTLDs):** .cn (China), .uk (United Kingdom), .fr (France), .ca (Canada), .jp (Japan)
- **Registry responsibility:**
 - Network Solutions → .com & .net
 - Educause → .edu

3. Authoritative Servers

- Provide **definitive information about specific domains.**
- Key Points:
 - **Organizational control** – maintained by organizations or service providers
 - **Hostname mapping** – provide authoritative hostname-to-IP address mapping
 - **Service flexibility** – can be self-managed or outsourced to DNS service providers

Role in DNS Resolution

When you type a URL in your browser:

1. Your device asks the **Local DNS Server**: “What is the IP of www.google.com?”
2. Local DNS checks its **cache**:
 -  If the IP is cached → sends it back immediately.
 -  If not cached → it becomes a **recursive resolver**, contacting other DNS servers on the internet:
 - **Root DNS Server** → points to TLD server (.com).
 - **TLD DNS Server** → points to Authoritative DNS for google.com.
 - **Authoritative DNS Server** → provides the final IP address.
3. The Local DNS server stores (caches) the answer for future requests.
4. It returns the IP to your computer → browser connects to the website.

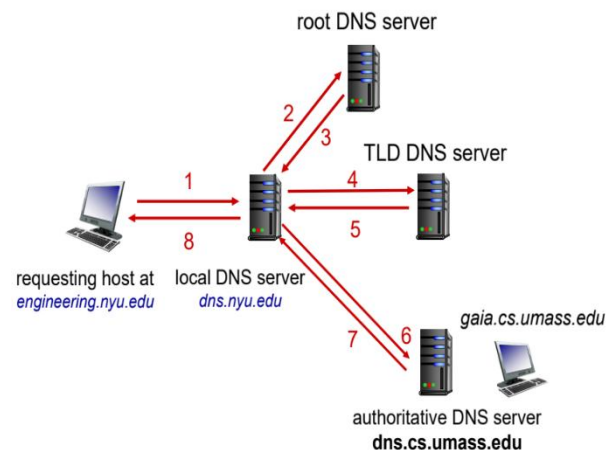
DNS Query Resolution Process

DNS supports two different query resolution methods, each with distinct characteristics and trade-offs.

Iterative Query Resolution: In iterative queries, the contacted server responds with the name of another server to contact rather than performing the query itself.

Iterative Query Process:

1. Client queries local DNS server
2. Local DNS server queries root server
3. Root server responds with TLD server name
4. Local DNS server queries TLD server
5. TLD server responds with authoritative server name
6. Local DNS server queries authoritative server
7. Authoritative server responds with IP address
8. Local DNS server returns result to client



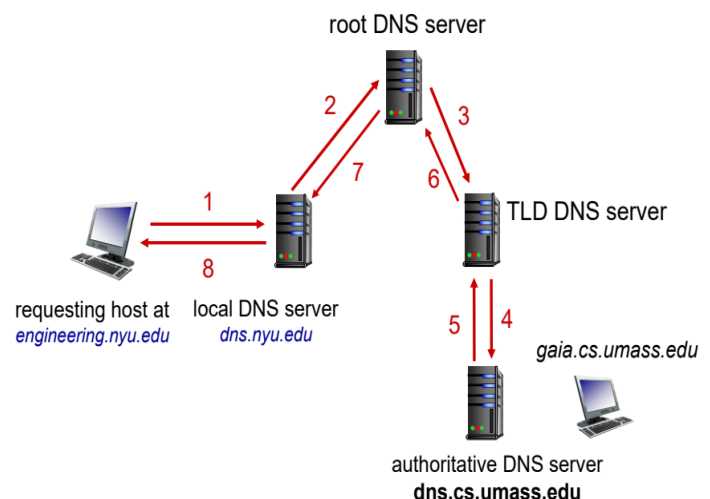
Iterative Query Characteristics:

- **Distributed load** - Query processing distributed across multiple servers
- **Client control** - Querying entity maintains control of query process
- **Reduced server load** - Individual servers don't handle full query resolution

Recursive Query Resolution: In recursive queries, each contacted server takes responsibility for resolving the query completely.

Recursive Query Process:

1. Client queries local DNS server
2. Local DNS server queries root server
3. Root server queries TLD server
4. TLD server queries authoritative server
5. Authoritative server responds to TLD server
6. TLD server responds to root server
7. Root server responds to local DNS server
8. Local DNS server responds to client



Recursive Query Characteristics:

- **Server burden** - Each server must complete the full query resolution
- **Hierarchical load** - Upper levels of hierarchy experience heavy query load
- **Client simplicity** - Clients only need to contact one server

DNS Caching and Performance

Caching plays a crucial role in DNS performance, reducing query latency and distributing load across the DNS infrastructure.

DNS Caching Mechanism: When any name server learns a mapping, it caches the information for future use. This caching dramatically improves performance and reduces load on authoritative servers.

Cache Management

- **TTL (Time To Live):** Every DNS record has a lifetime (e.g., 300 seconds = 5 mins). After this, the cache entry expires, and a new lookup is done.
- **Cache hierarchy:**
 - Root servers (.) rarely change → cached for a long time.
 - TLD servers (.com, .org, etc.) also cached in local DNS servers.
 - Only actual domain records (like www.example.com) expire quickly.
- **Root server optimization:** Because of caching, we **don't ask root servers often** → they stay less loaded.
- **Update challenges:** If the owner of a domain changes their IP, the old IP may stay in caches until TTL ends.

Cache Consistency Issues

- **IP address changes:** Suppose example.com moves from 192.168.1.1 → 192.168.1.2.
 - If TTL is 24 hours, users may still get the **old IP** for up to a day.
- **DNS propagation delay:** That's why when websites change hosting, it may take **hours/days** for everyone worldwide to see the new site.
- **Best-effort translation:** DNS does not guarantee "perfectly updated info." It only tries its best, based on TTL and caching.

DNS Records and Resource Records

DNS stores information in standardized Resource Records (RRs), each containing specific types of information about domain names and network resources.

Resource Record Format: Each RR contains four fields: (name, value, type, ttl)

DNS Record Types:

A Records (Address):

- **Purpose** - Map hostname to IP address
- **Name field** - Hostname (e.g., www.example.com)
- **Value field** – IP address (e.g., 192.168.1.1)

NS Records (Name Server):

- **Purpose** - Identify authoritative name servers for a domain
- **Name field** - Domain name (e.g., foo.com)
- **Value field** - Hostname of authoritative name server

CNAME Records (Canonical Name):

- **Purpose** - Create aliases for existing hostnames
- **Name field** - Alias name
- **Value field** - Canonical (real) name
- **Example** - www.ibm.com might be an alias for servereast.backup2.ibm.com

MX Records (Mail Exchange):

- **Purpose** - Identify mail servers for a domain
- **Name field** - Domain name
- **Value field** - Mail server hostname

DNS Security

DNS security has become increasingly important as cyber attacks targeting DNS infrastructure have grown more sophisticated.

DNS Security Threats:

1. Distributed Denial of Service (DDoS) Attacks

- **Root server attacks:** Attackers flood root DNS servers with huge amounts of traffic to make them unreachable.
- **Mitigation strategies:** Use of **traffic filtering**, **load balancing**, and **extensive caching** at local resolvers reduces impact.
- **TLD server attacks:** Targeting Top-Level Domain servers (e.g., .com, .org) is more dangerous, since disruption affects millions of domains.
- **Amplification attacks:** Attackers use open DNS resolvers to send small queries that trigger **large responses**, amplifying the attack traffic towards a victim.

2. DNS Poisoning and Spoofing

- **Cache poisoning:** Inserting **false DNS records** into a resolver's cache so that users are redirected to malicious sites.
- **Man-in-the-middle attacks:** An attacker intercepts DNS queries and modifies responses before they reach the user.

- **Response spoofing:** An attacker forges fake DNS responses, tricking the client into believing it came from the legitimate DNS server.

DNSSEC (DNS Security Extensions)

DNSSEC adds security to the DNS system by ensuring that DNS responses are authentic and not tampered with.

Key Features:

- **Digital Signatures:** Every DNS response is signed with a private key, so the resolver can verify it with the public key.
- **Chain of Trust:** Verification is hierarchical — starts from the **root**, then **TLD (.com, .org, etc.)**, and finally the **authoritative server**.
- **Data Integrity:** Ensures the DNS record wasn't altered during transit.
- **Authentication:** Confirms the DNS response really came from the correct authoritative server.

8. Peer-to-Peer Applications

P2P Architecture Principles

Unlike client-server models, P2P systems avoid dependence on central servers by distributing control and resources among peers.

Core P2P Characteristics

- **No always-on server:** Unlike client-server systems, P2P networks do not rely on a central server.
- **Direct communication:** End systems (peers) communicate directly with each other.
- **Dual role of peers:** Each peer acts as both a **client** (requesting services) and a **server** (providing services).
- **Resource sharing:** CPU, storage, and bandwidth are shared among participants.
- **Distributed system:** Control and data are spread across many peers, reducing single points of failure.
- **Fault tolerance:** Failure of individual peers has limited impact due to distributed nature.

Self-Scalability

- **Inherent advantage:** As more peers join, they bring additional **resources** (processing power, storage, bandwidth).
- **Balanced growth:** New peers also create **extra demand**, but the system scales naturally because supply and demand grow together.
- **Result:** The system can handle more users without becoming overloaded.

P2P Challenges:

- **Intermittent connectivity** - Peers can join and leave the network unpredictably
- **Dynamic addressing** - Changing IP addresses complicate peer identification
- **Decentralized management** - No central authority to coordinate activities
- **Security concerns** - Distributed nature makes security enforcement difficult
- **Quality of service** - Difficult to guarantee consistent performance

P2P Applications: Modern P2P applications include BitTorrent for file sharing, Skype for VoIP communication, streaming applications like KanKan, and various distributed computing projects.

File Distribution Analysis

Understanding the performance characteristics of different distribution models is crucial for application design and deployment decisions.

Distribution Problem Definition: Consider distributing a file of size F from one server to N peers, where each peer has upload capacity u_i and download capacity d_i , and the server has upload capacity u_s .

System Setup

- File size = F , Server upload rate = u_s , Peer i upload rate = u_i , Peer i download rate = d_i , $d_{min} = \min(d_i) \rightarrow$ slowest peer download rate

1. Client-Server File Distribution

- The server must **upload N copies** of the file (one to each peer).
 - Time for server upload:

$$T_{server} = \frac{N \cdot F}{u_s}$$

- Each peer must download the file:
 - Time for slowest peer to finish:

$$T_{client} = \frac{F}{d_{min}}$$

- Total distribution time:

$$D_{c-s} \geq \max \left\{ \frac{N \cdot F}{u_s}, \frac{F}{d_{min}} \right\}$$

 **Observation:** In client-server, time **increases linearly with N** , since the server must send N copies.

2. Peer-to-Peer (P2P) File Distribution

- The server must upload at least one full copy of the file:

$$T_{server} = \frac{F}{u_s}$$

- Each peer must download one full copy:

$$T_{client} = \frac{F}{d_{min}}$$

- The **system as a whole** must transfer $N \times F$ bits (one copy per peer).

- Aggregate upload capacity =

$$u_s + \sum_{i=1}^N u_i$$

- Time needed:

$$T_{aggregate} = \frac{N \cdot F}{u_s + \sum_{i=1}^N u_i}$$

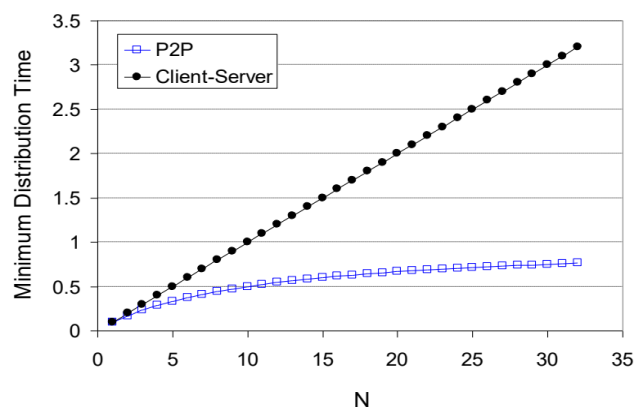
- Total distribution time:

$$D_{P2P} \geq \max \left\{ \frac{F}{u_s}, \frac{F}{d_{min}}, \frac{N \cdot F}{u_s + \sum_{i=1}^N u_i} \right\}$$

Observation: In P2P, distribution time grows with N, but so does system capacity (since each peer contributes upload bandwidth). **Thus, P2P scales much better than client-server.**

Performance Comparison: Client-Server vs P2P

- Client-Server:**
 - Distribution time increases linearly with the number of clients (N).
 - Reason:** The server must upload one full copy per client, so more clients → more time.
- P2P:**
 - Distribution time remains relatively stable even as N grows.
 - Reason:** Each new peer contributes additional upload bandwidth, which helps serve the file to others.

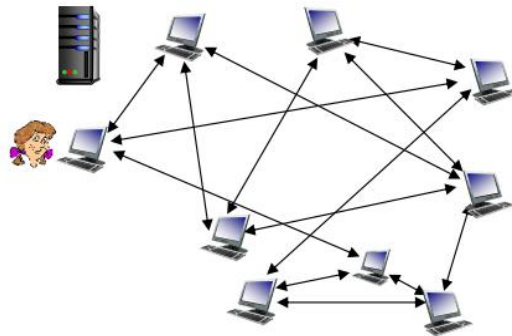


BitTorrent Protocol

BitTorrent represents one of the most successful P2P file sharing protocols, demonstrating effective solutions to many P2P design challenges.

BitTorrent Architecture:

Torrent Concept: A torrent represents a group of peers exchanging chunks of the same file. The file is divided into fixed-size chunks (typically 256KB), and peers exchange these chunks until each peer has the complete file.



Tracker Functionality: The tracker maintains a list of peers participating in each torrent. When a peer wants to join a torrent, it contacts the tracker to obtain a list of other participating peers.

Peer Participation Process:

1. **Joining** - New peer contacts tracker and receives peer list
2. **Connection** - Peer establishes connections with subset of available peers
3. **Exchange** - Peer downloads and uploads chunks with connected peers
4. **Completion** - Peer may leave or remain in torrent after obtaining complete file

Chunk Request Strategy: BitTorrent divides a file into many **chunks (pieces)**. Peers download chunks from each other and also upload to others.

Rarest First Strategy:

- Each peer **requests the least common chunks first** (the ones few peers currently have).
- This prevents the system from ending up in a situation where:
 - Many peers have the **same popular chunks**, but
 - Some **rare chunks** exist in only one or two peers (risking unavailability if they leave).

Rarest First Benefits:

- **Chunk diversity** - Maintains good distribution of all file chunks
- **System efficiency** - Prevents bottlenecks caused by rare chunk scarcity
- **Robustness** - Reduces risk of chunk unavailability due to peer departure

Tit-for-Tat Algorithm: BitTorrent uses a tit-for-tat reciprocation algorithm to encourage peers to contribute upload capacity.

Reciprocation Mechanism:

- **Top four providers** - Each peer uploads to the four peers providing the highest download rates
- **Choking** - Peers not in the top four are "choked" (don't receive uploads)
- **Regular reevaluation** - Top four list updated every 10 seconds based on current performance
- **Optimistic unchoking** - Every 30 seconds, randomly select one additional peer for upload

Optimistic Unchoking Benefits:

- **Peer discovery** - Allows evaluation of new peers' upload capabilities
- **Bootstrap assistance** - Helps new peers obtain their first chunks
- **System exploration** - Prevents lock-in to suboptimal peer relationships

BitTorrent Incentive System: The tit-for-tat mechanism creates strong incentives for peers to contribute upload capacity. Peers that provide faster uploads receive faster downloads, creating a natural market mechanism that encourages participation and contribution.

9. Video Streaming and CDNs

Video Streaming Context and Challenges

Video streaming has become the dominant consumer of Internet bandwidth, with services like Netflix, YouTube, and Amazon Prime accounting for approximately 80% of residential ISP traffic as of 2020. This massive scale creates significant technical and logistical challenges.

Scale Challenges: The fundamental challenge is reaching approximately one billion users simultaneously while maintaining acceptable quality of service. A single centralized server cannot handle this scale due to bandwidth limitations, geographic distance, and reliability concerns.

Heterogeneity Challenges: Users have vastly different capabilities and constraints:

- **Device variations** - Mobile devices vs. desktop computers vs. smart TVs
- **Connection types** - Wired broadband vs. mobile networks vs. satellite connections
- **Bandwidth availability** - High-speed fiber vs. limited mobile data plans
- **Geographic distribution** - Users scattered across the globe with varying infrastructure quality

Solution Approach: The solution involves distributed, application-level infrastructure that brings content closer to users while adapting to their specific capabilities and constraints.

Video Encoding and Compression

Understanding video representation and compression is essential for streaming system design.

Digital Video Fundamentals: Video consists of sequences of images (frames) displayed at constant rates, typically 24, 30, or 60 frames per second. Each frame is composed of pixels, with each pixel represented by digital bits indicating color and intensity values.

Video Compression Techniques:

Spatial Compression: Spatial compression reduces redundancy within individual frames by identifying and eliminating repetitive visual patterns.

- **Color repetition** - Instead of storing N identical color values, store one color value and repetition count
- **Pattern recognition** - Identify and compress repetitive visual elements
- **Lossy compression** - Accept some quality loss for significant size reduction

Temporal Compression: Temporal compression reduces redundancy between consecutive frames by storing only the differences between frames.

- **Frame differences** - Store changes from one frame to the next rather than complete frames
- **Motion vectors** - Track object movement between frames
- **Predictive coding** - Use previous frames to predict future frame content

Video Encoding Standards:

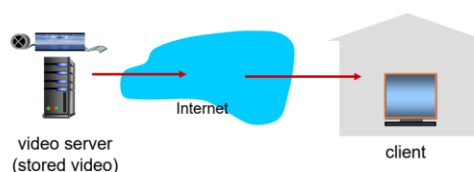
- **MPEG-1** - Designed for CD-ROM, typically 1.5 Mbps
- **MPEG-2** - DVD quality, 3-6 Mbps
- **MPEG-4** - Internet-optimized, 64 Kbps to 12 Mbps
- **H.264/AVC** - High efficiency, widely used for streaming
- **H.265/HEVC** - Next-generation compression for 4K content

Bit Rate Strategies:

- **Constant Bit Rate (CBR)** - Fixed encoding rate regardless of content complexity
- **Variable Bit Rate (VBR)** - Encoding rate varies based on content complexity and motion

Streaming Protocols and Techniques

Basic Streaming Model: In simple streaming scenarios, a video server transmits stored video content over the Internet to clients who play the content in real-time. However, network variability creates significant challenges for maintaining smooth playback.



Network Variability Challenges:

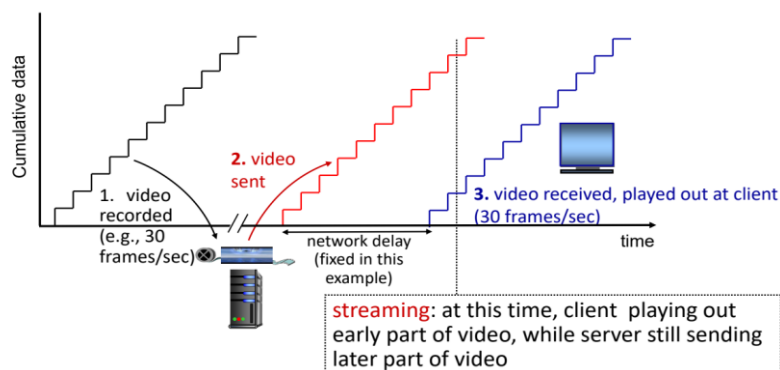
- **Bandwidth fluctuations** - Available bandwidth changes due to network congestion
- **Packet loss** - Network congestion can cause packet loss, affecting video quality
- **Jitter** - Variable network delays disrupt smooth playback timing
- **Geographic distance** - Long network paths introduce additional delays and variability

Client-Side Buffering: To compensate for network variability, clients implement playout buffering systems.

Streaming Stored Video

Process

1. Video is **recorded**.
2. Video is **sent** from server to client.
3. Client **receives and plays out** video at a fixed rate (e.g., 30 frames/sec).
 - While the client plays early parts, the server is still sending later parts.



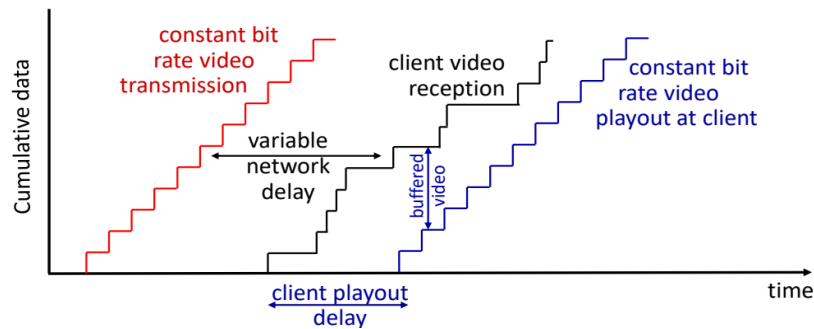
Challenges

- **Continuous playback constraint:** Once playback begins, frames must be played at the correct rate (timing of original video).
- **Variable network delays (jitter):** Can disrupt smooth playback.
- **Need for buffering:** Client maintains a buffer to absorb delay variations.
- **Client interactivity:** Users expect pause, fast-forward, rewind, or jumping to sections.
- **Packet loss:** Some video packets may be lost or retransmitted, affecting playback quality.

Playout Buffering

- The server sends video at a steady rate.
- But the **network may delay some packets**.
- To fix this, the client **buffers data** and starts playback a little later.

- This **playout delay** helps keep playback smooth even if packets arrive late.



Dynamic Adaptive Streaming over HTTP (DASH)

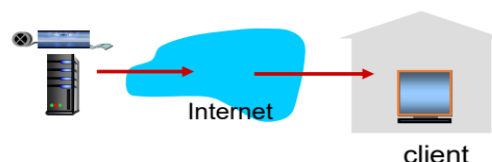
DASH represents a modern approach to video streaming that adapts to network conditions and user capabilities.

DASH Architecture:

Server-Side Components:

- **Multiple encodings** - Videos encoded at different bit rates and resolutions
- **Chunk division** - Each encoding divided into time-based chunks (typically 2-10 seconds)
- **Manifest file** - Contains URLs and metadata for all available chunks and encodings

Client-Side Intelligence: DASH places adaptation logic at the client, enabling intelligent decisions based on local conditions.



Client Decision Making:

- **Bandwidth measurement** - Continuous monitoring of available bandwidth
- **Quality selection** - Choose appropriate encoding based on current conditions
- **Buffer management** - Decide when to request next chunk based on buffer status
- **Server selection** - Choose optimal server location for each chunk request

Adaptive Streaming Benefits:

- **Quality optimization** - Clients receive highest quality sustainable by their connection
- **Resilience** - Automatic adaptation to network condition changes
- **Efficiency** - Reduces bandwidth waste on unnecessary quality

- **User experience** - Minimizes playback interruptions and startup delays

Content Delivery Networks (CDNs)

A **Content Delivery Network (CDN)** is a **distributed network of servers** placed at different geographical locations.

Its purpose is to **deliver web content (like images, videos, scripts, and entire websites) quickly and reliably** to users, by serving content from the nearest server instead of the origin server.

◆ How It Works

1. Origin Server

- The main server where the website/application content is hosted.

2. Edge Servers (CDN Servers)

- Cached copies of the content are stored in many servers located worldwide.
- These are called **PoPs (Points of Presence)**.

3. User Request Flow

- User requests a webpage.
- Instead of fetching from the faraway origin, the request is redirected to the **nearest CDN server**.
- If content is cached → served immediately.
- If not cached → fetched from origin → cached at edge → served to user.

◆ Example

- Without CDN:
User in Bangladesh requests a US-hosted website → must travel thousands of km → high latency.
- With CDN:
Same request served from a local CDN node in Dhaka or Singapore → much faster.

Socket Programming

Definition:

Socket programming is a way for **two programs (on the same or different computers)** to communicate over a network. A **socket** is an endpoint for sending or receiving data.

Key Concepts

1. **Socket:** A combination of an **IP address** and a **port number** used to identify a network connection.
2. **Server and Client:**
 - **Server:** Waits for incoming connections.
 - **Client:** Initiates the connection to the server.
3. **Types of Sockets:**
 - **TCP (Stream) Socket:** Reliable, connection-oriented, ensures all data is delivered in order.
 - **UDP (Datagram) Socket:** Faster, connectionless, no guarantee of delivery or order.